

Assignment: Portfolio Choice, Diversification and CAPM Evaluation

Introduction to Quantitative Finance and Risk Management — MSc in Data Science

- **Due Date:** July 2nd, 2026
 - **Weight:** 20% of final course grade
 - **Programming Language:** Python
 - **Submission Format:** Jupyter Notebook `.ipynb` + PDF export + Python code file `.py` or zipped code folder
-

Objective

Apply the tools developed in Sessions 1–4 to a real investment problem. Construct portfolios using historical stock returns, examine the benefits and limits of diversification, compute efficient portfolios, and evaluate their out-of-sample performance. Connect portfolio choice to the Capital Asset Pricing Model by estimating beta, expected return and abnormal performance.

The assignment combines the following topics:

- realized and expected stock returns
 - variance, covariance and correlation
 - diversification
 - mean–variance portfolio choice
 - the efficient frontier
 - the minimum-variance portfolio
 - the tangency portfolio
 - the risk-free asset and Sharpe ratio
 - beta and the CAPM
 - out-of-sample portfolio evaluation
-

1. Data Collection

Download **weekly adjusted closing prices** from Yahoo Finance using `yfinance` or an equivalent Python library.

Use:

- **10 listed stocks** of your choice
- **one broad market index or ETF** as the market proxy (e.g. `SPY`, `^GSPC`, or another well-justified benchmark)

Dataset should cover the period **1 January 2016 to 31 March 2026**:

- **Estimation period:** 1 January 2016 to 31 December 2025
- **Out-of-sample evaluation period:** 1 January 2026 to 31 March 2026

Requirements:

- Briefly **justify your stock selection**. The portfolio should **not consist only of firms from the same sector**.
 - If you cannot obtain a reliable risk-free rate series, you may assume a **risk-free rate of 0%** — but state this clearly.
-

2. Return Construction and Descriptive Statistics

Compute **weekly simple returns** for all assets.

For the **estimation period**, report for each stock:

- mean weekly return
- annualized mean return
- weekly volatility
- annualized volatility
- minimum weekly return
- maximum weekly return

Annualize returns and volatilities using **52 weeks**.

Also compute and present:

- the **covariance matrix**
- the **correlation matrix**

Include:

- a **table of descriptive statistics**
 - a clearly formatted **correlation matrix or correlation heatmap**
 - a short **discussion** of which assets appear most volatile and which pairs of assets seem to offer the strongest diversification benefits
-

3. Diversification Analysis

Construct two simple portfolios:

1. an **equally weighted portfolio** of the 10 stocks
2. a **concentrated portfolio** consisting only of the **2 most volatile stocks**

For both portfolios, calculate:

- expected annualized return
- annualized volatility

- Sharpe ratio
- cumulative performance during the estimation period

Plot the cumulative performance of both portfolios over the estimation period.

Discuss: Does holding more stocks automatically reduce risk? Explain using the correlation matrix and computed portfolio volatilities.

4. Efficient Frontier Without Short-Selling

Construct the mean–variance efficient frontier using only the 10 stocks, assuming:

$$w_i \geq 0 \text{ and } \sum_{i=1}^N w_i = 1$$

i.e. **short-selling is not allowed** and the portfolio must be **fully invested**.

On the efficient frontier plot, clearly mark:

- the individual stocks
- the equally weighted portfolio
- the minimum-variance portfolio
- the tangency portfolio (the portfolio with the **highest Sharpe ratio**)

If assuming a risk-free rate of 0%, state this clearly. Otherwise, explain how the risk-free rate is estimated or proxied.

Report the portfolio weights of the **minimum-variance portfolio** and the **tangency portfolio**.

Discuss: Which assets receive the largest weights? Are the weights economically intuitive? Are any assets excluded from the optimized portfolios? Why might this happen?

5. Efficient Frontier With Bounded Short-Selling

Repeat the efficient frontier calculation allowing **bounded short-selling**, assuming:

$$-1 \leq w_i \leq 1 \text{ and } \sum_{i=1}^N w_i = 1$$

Overlay on the same graph:

1. the no-short-selling efficient frontier
2. the bounded-short-selling efficient frontier

Discuss: How does allowing short-selling change the efficient frontier? Does the tangency portfolio become more extreme? Why can short-selling improve the mathematical frontier but increase practical investment risk?

6. CAPM and Beta Estimation

Using the market proxy, estimate each stock's beta over the estimation period via the regression:

$$R_{i,t} - R_{f,t} = \alpha_i + \beta_i (R_{M,t} - R_{f,t}) + \varepsilon_{i,t}$$

where $R_{i,t}$ is the return on stock i , $R_{M,t}$ is the return on the market proxy, and $R_{f,t}$ is the risk-free rate. If assuming $R_f = 0$, the regression becomes $R_{i,t} = \alpha_i + \beta_i R_{M,t} + \varepsilon_{i,t}$.

For each stock, report:

- beta
- alpha
- average realized annualized return
- CAPM-implied expected return
- difference between realized return and CAPM-implied return

Then compute the **beta** of:

- the equally weighted portfolio
- the minimum-variance portfolio
- the tangency portfolio

Create a **Security Market Line (SML) plot** with:

- beta on the x-axis
- average annualized return on the y-axis
- the CAPM line
- the individual stocks marked as points

Discuss: Which stocks have the highest and lowest market exposure? Does the tangency portfolio have a high or low beta? Do high-beta stocks in your sample earn higher average returns? Does your evidence support the CAPM prediction?

7. Out-of-Sample Evaluation

Using the portfolio weights estimated **as of 31 December 2025**, evaluate the following portfolios during the **out-of-sample period (1 January 2026 to 31 March 2026)**:

1. equally weighted portfolio
2. minimum-variance portfolio
3. tangency portfolio
4. bounded-short-selling tangency portfolio (if applicable)

For each portfolio, calculate:

- cumulative return
- annualized volatility
- Sharpe ratio
- maximum drawdown
- beta relative to the market proxy
- Jensen's alpha (if applicable)

Plot the cumulative return trajectories of all portfolios and the market benchmark over the out-of-sample period.

Discuss: Did the optimized portfolios outperform the equally weighted portfolio? Was the tangency portfolio stable out of sample? Did the minimum-variance portfolio actually deliver lower realized risk? What does this tell us about estimation error in mean–variance optimization?

8. Final Discussion

Conclude the notebook with a short discussion of **no more than 500 words** addressing:

- what you learned about diversification
 - whether mean–variance optimization produced sensible portfolios
 - whether your optimized portfolios performed well out of sample
 - whether your results support the CAPM
 - the main limitations of your analysis
-

9. Deliverables

You must submit:

1. a well-documented Jupyter Notebook
2. a PDF export of the notebook
3. the Python code file or zipped code folder
4. any manually downloaded data files (if used)

The notebook should include:

- clean and readable code
 - comments explaining the main steps
 - tables and graphs
 - interpretation after each major result
 - a short final conclusion
-

10. Suggested Python Libraries

```
import yfinance as yf
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import minimize
import statsmodels.api as sm

# Optional
import seaborn as sns
```

11. Marking Scheme

Section	Weight
Data collection, cleaning and return construction	10%
Descriptive statistics and diversification analysis	15%
Efficient frontier without short-selling	20%
Efficient frontier with bounded short-selling	10%
CAPM, beta and Security Market Line analysis	20%
Out-of-sample portfolio evaluation	15%
Interpretation, presentation and code quality	10%
Total	100%

12. Important Notes

- Results should be **reproducible** — the notebook must run from beginning to end without errors.
 - Do **not** only present code and graphs — you must **explain your results**.
 - The **quality of interpretation is as important as the numerical results**. In particular, be able to explain *why optimized portfolios may perform poorly out of sample, even when they look attractive in the estimation period*.
-

Analysis

The cell above is the assignment brief for reference. The notebook below implements it section by section. Run top-to-bottom; it is fully reproducible (fixed tickers, dates and a stated risk-free assumption).

0. Setup & Configuration

```
#
=====
=====
# Imports, global configuration and reproducibility
#
=====
=====
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import yfinance as yf
import statsmodels.api as sm
from scipy.optimize import minimize
import seaborn as sns

# --- Plotting defaults ---
sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (10, 6)
plt.rcParams["figure.dpi"] = 110

# --- Reproducibility ---
np.random.seed(42)

# --- Universe: one stock per GICS sector (10 of 11 sectors) ---
TICKERS = {
    "LRCX": "Lam Research (Information Technology)",
    "DE": "Deere & Co. (Industrials)",
    "VRTX": "Vertex Pharmaceuticals (Health Care)",
    "LULU": "Lululemon (Consumer Discretionary)",
    "EOG": "EOG Resources (Energy)",
    "SPGI": "S&P Global (Financials)",
    "NEE": "NextEra Energy (Utilities)",
    "FCX": "Freeport-McMoRan (Materials)",
    "TTWO": "Take-Two Interactive (Communication Services)",
    "COST": "Costco (Consumer Staples)",
}

STOCKS = list(TICKERS.keys())
MARKET = "SPY" # market proxy / benchmark
ALL_SYMBOLS = STOCKS + [MARKET]

# --- Sample windows ---
```

```

START_DATE = "2016-01-01"
SPLIT_DATE = "2025-12-31"           # end of estimation period
(inclusive)
OOS_START   = "2026-01-01"         # start of out-of-sample period
END_DATE    = "2026-03-31"         # end of out-of-sample period

# --- Conventions ---
RF_ANNUAL    = 0.0                 # risk-free rate assumed 0% (stated
explicitly)
RF_WEEKLY    = RF_ANNUAL / 52
WEEKS_PER_YEAR = 52               # annualization factor

print(f"{len(STOCKS)} stocks + market proxy '{MARKET}'")
print(f"Estimation    : {START_DATE} -> {SPLIT_DATE}")
print(f"Out-of-sample: {OOS_START} -> {END_DATE}")
print(f"Risk-free rate assumed: {RF_ANNUAL:.0%}")

10 stocks + market proxy 'SPY'
Estimation    : 2016-01-01 -> 2025-12-31
Out-of-sample: 2026-01-01 -> 2026-03-31
Risk-free rate assumed: 0%

#
=====
=====
# Required Libraries Installation
#
=====
=====
# Uncomment and run the following lines if libraries are not
installed:

# !pip install numpy           # Numerical computing
# !pip install pandas          # Data manipulation and analysis
# !pip install matplotlib      # Plotting and visualization
# !pip install yfinance        # Yahoo Finance data download
# !pip install statsmodels     # Statistical models and regression
# !pip install scipy           # Scientific computing and optimization
# !pip install seaborn         # Statistical data visualization

```

1. Data Collection

Stock selection & justification. Our guiding principle was *maximum sector diversification combined with full data availability*. Instead of the usual cluster of mega-cap technology names, we selected one representative company from **ten of the eleven GICS sectors**, so that no single industry dominates the portfolio and the correlation structure is as rich as possible — precisely the setting in which the benefits of diversification and mean–variance optimization are most visible. This also directly satisfies the requirement that the portfolio not consist only of firms from the same sector.

Ticker	Company	GICS Sector
LRCX	Lam Research	Information Technology
DE	Deere & Co.	Industrials
VRTX	Vertex Pharmaceuticals	Health Care
LULU	Lululemon	Consumer Discretionary
EOG	EOG Resources	Energy
SPGI	S&P Global	Financials
NEE	NextEra Energy	Utilities
FCX	Freeport-McMoRan	Materials
TTWO	Take-Two Interactive	Communication Services
COST	Costco	Consumer Staples

Every name has been listed since well before 2016, which guarantees a complete weekly price history across the entire estimation window with no survivorship gaps to patch. We also deliberately balanced the risk profile: high-volatility cyclicals such as Freeport-McMoRan (copper mining), Lam Research (semiconductor equipment) and Lululemon (consumer discretionary) sit alongside defensive, lower-beta compounds like NextEra Energy, Costco and S&P Global. This spread ensures that the diversification analysis, the efficient frontier and the CAPM/beta estimation all produce meaningful, contrasting results rather than ten near-identical return series.

Market proxy: SPY (S&P 500 ETF) — a liquid, tradable benchmark with clean dividend-adjusted prices.

Risk-free rate: assumed **0%** throughout, as permitted by the brief. This is stated explicitly and simplifies the Sharpe ratio and the CAPM regression (which becomes $R_{i,t} = \alpha_i + \beta_i R_{M,t} + \varepsilon_{i,t}$).

```
# --- Download weekly adjusted closing prices from Yahoo Finance ---
# auto_adjust=True -> the 'Close' column is already adjusted for
# splits & dividends.
raw = yf.download(
    ALL_SYMBOLS,
    start=START_DATE,
    end=END_DATE,
    interval="1wk",
    auto_adjust=True,
    progress=False,
)

prices = raw["Close"][ALL_SYMBOLS].copy()
prices = prices.dropna(how="all").sort_index()

# --- Sanity checks ---
print("Price matrix shape:", prices.shape)
print("Date range:", prices.index.min().date(), "->",
      prices.index.max().date())
print("\nMissing values per column:")
```

```
print(prices.isna().sum())
```

```
prices.tail()
```

```
Price matrix shape: (535, 11)
```

```
Date range: 2016-01-01 -> 2026-03-27
```

```
Missing values per column:
```

```
Ticker
```

```
LRCX      0
```

```
DE        0
```

```
VRTX      0
```

```
LULU      0
```

```
EOG        0
```

```
SPGI       0
```

```
NEE        0
```

```
FCX        0
```

```
TTWO       0
```

```
COST       0
```

```
SPY        0
```

```
dtype: int64
```

Ticker	LRCX	DE	VRTX	LULU	EOG
--------	------	----	------	------	-----

\

Date

2026-02-27	214.423096	588.967407	460.820007	173.179993	130.020477
------------	------------	------------	------------	------------	------------

2026-03-06	209.490005	584.121582	478.130005	158.190002	132.014984
------------	------------	------------	------------	------------	------------

2026-03-13	233.990005	565.924805	458.049988	165.570007	137.750473
------------	------------	------------	------------	------------	------------

2026-03-20	211.619995	579.495117	453.739990	151.389999	146.353668
------------	------------	------------	------------	------------	------------

2026-03-27	213.660004	561.657227	446.540009	153.100006	143.456161
------------	------------	------------	------------	------------	------------

Ticker	SPGI	NEE	FCX	TTWO	COST
--------	------	-----	-----	------	------

\

Date

2026-02-27	450.062347	89.854683	62.522327	212.529999	981.146362
------------	------------	-----------	-----------	------------	------------

2026-03-06	418.725311	91.063011	59.079906	209.199997	1001.866272
------------	------------	-----------	-----------	------------	-------------

2026-03-13	425.150360	91.738068	53.502190	201.750000	973.367615
------------	------------	-----------	-----------	------------	------------

2026-03-20	411.492157	90.497162	55.707336	194.720001	978.230591
------------	------------	-----------	-----------	------------	------------

2026-03-27	424.352203	92.204643	58.650852	197.500000	994.986206
------------	------------	-----------	-----------	------------	------------

Ticker	SPY
Date	
2026-02-27	679.454407
2026-03-06	664.245911
2026-03-13	658.002991
2026-03-20	643.333069
2026-03-27	650.340027

2. Return Construction and Descriptive Statistics

We compute **weekly simple returns** for every asset, split the sample into the estimation and out-of-sample windows, then report descriptive statistics, the covariance matrix and the correlation matrix **over the estimation period only**. Returns/volatilities are annualized with 52 weeks.

```
# --- Weekly simple returns for every asset ---
returns = prices.pct_change().dropna(how="any")

# --- Split into estimation and out-of-sample windows ---
ret_est = returns.loc[:SPLIT_DATE]          # 2016 -> 2025-12-31
ret_oos = returns.loc[OOS_START:]          # 2026-01-01 -> 2026-03-31

# Convenience views
ret_est_stocks = ret_est[STOCKS]
ret_est_mkt     = ret_est[MARKET]
ret_oos_stocks = ret_oos[STOCKS]
ret_oos_mkt     = ret_oos[MARKET]

print("Estimation weeks:", len(ret_est), "| Out-of-sample weeks:",
      len(ret_oos))
ret_est_stocks.head()
```

Estimation weeks: 521 | Out-of-sample weeks: 13

Ticker \ Date	LRCX	DE	VRTX	LULU	EOG	SPGI
2016-01-08	0.003347	-0.018536	-0.113518	0.033583	-0.003055	-0.068262
2016-01-15	-0.053232	-0.007744	-0.067118	0.008123	-0.020523	-0.036867
2016-01-22	0.019525	0.024374	-0.024298	0.043966	0.076549	0.007460
2016-01-29	0.029086	0.051998	-0.011212	0.050839	-0.002768	0.056081
2016-02-05	-0.101581	-0.029097	-0.130942	-0.086380	-0.042081	-0.054483

Ticker	NEE	FCX	TTW0	COST
Date				
2016-01-08	0.009456	-0.251337	0.008328	-0.010593
2016-01-15	-0.006150	0.030953	0.008565	-0.018279
2016-01-22	0.031036	0.020785	0.019412	-0.005254
2016-01-29	0.048199	0.294118	0.043737	-0.042182
2016-02-05	-0.014271	-0.145105	-0.000285	0.023520

--- Per-stock descriptive statistics over the estimation period ---

```
desc = pd.DataFrame({
    "Mean weekly": ret_est_stocks.mean(),
    "Ann. mean":   ret_est_stocks.mean() * WEEKS_PER_YEAR,
    "Weekly vol":  ret_est_stocks.std(),
    "Ann. vol":    ret_est_stocks.std() * np.sqrt(WEEKS_PER_YEAR),
    "Min weekly":  ret_est_stocks.min(),
    "Max weekly":  ret_est_stocks.max(),
})
desc.sort_values("Ann. vol", ascending=False).round(4)
```

	Mean weekly	Ann. mean	Weekly vol	Ann. vol	Min weekly	Max weekly
FCX	0.0071	0.3703	0.0743	0.5355	-0.2823	
0.4622						
LRCX	0.0080	0.4139	0.0566	0.4082	-0.2049	
0.3350						
LULU	0.0041	0.2151	0.0556	0.4009	-0.2532	
0.2967						
EOG	0.0031	0.1613	0.0549	0.3957	-0.4747	
0.2217						
VRTX	0.0036	0.1888	0.0447	0.3224	-0.1983	
0.2274						
TTW0	0.0049	0.2525	0.0421	0.3036	-0.1695	
0.1475						
DE	0.0046	0.2398	0.0399	0.2874	-0.1803	
0.1624						
NEE	0.0032	0.1676	0.0335	0.2417	-0.2436	
0.1159						
SPGI	0.0041	0.2126	0.0331	0.2385	-0.2046	
0.1789						
COST	0.0041	0.2130	0.0302	0.2180	-0.1325	
0.0994						

--- Covariance and correlation matrices (weekly, estimation period)

```
---
cov_matrix = ret_est_stocks.cov()
corr_matrix = ret_est_stocks.corr()

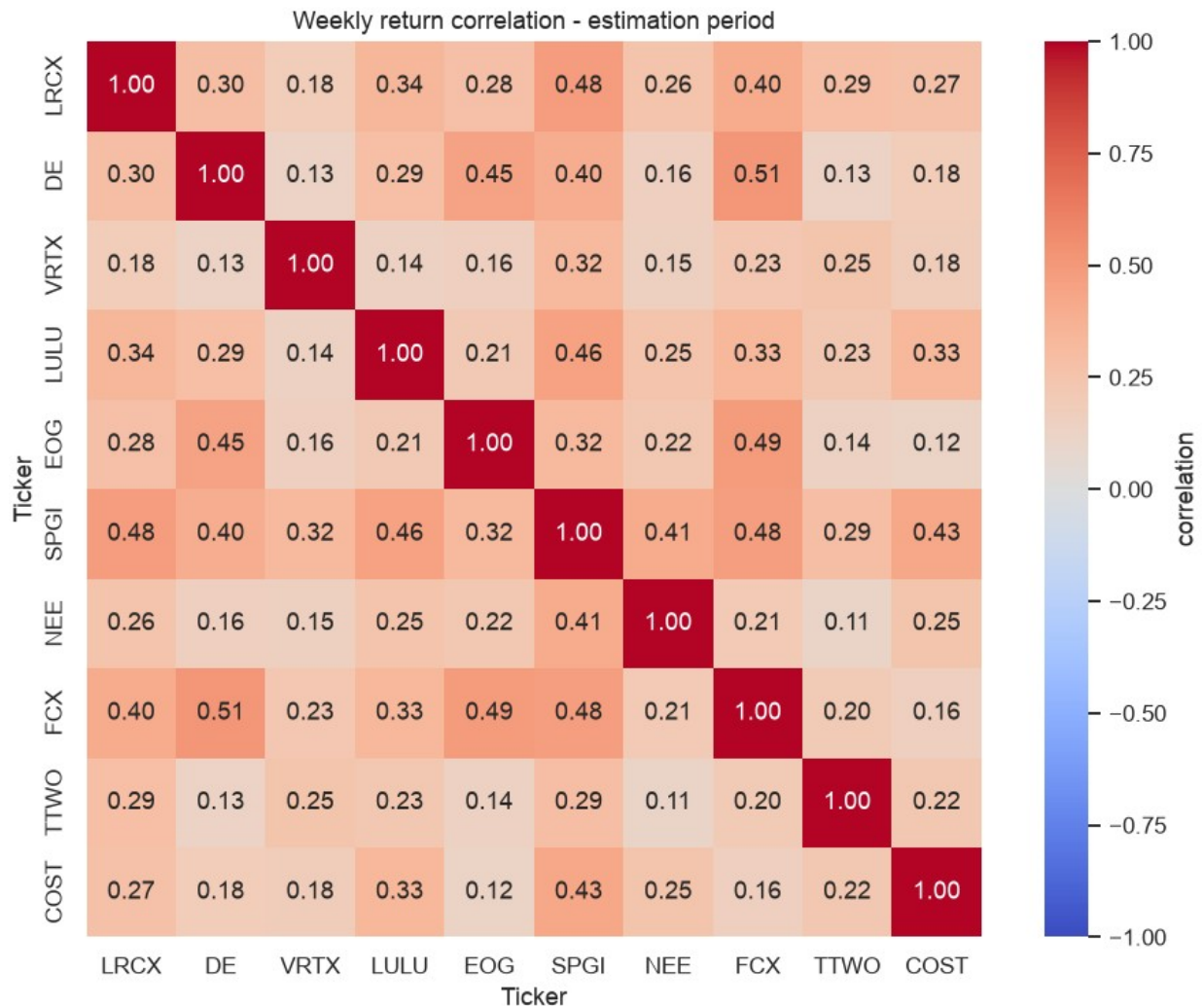
fig, ax = plt.subplots(figsize=(9, 7))
```

```

sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap="coolwarm",
            vmin=-1, vmax=1, square=True, cbar_kws={"label":
"correlation"}, ax=ax)
ax.set_title("Weekly return correlation - estimation period")
plt.tight_layout()
plt.show()

cov_matrix.round(5)

```



Ticker \ Ticker	LRCX	DE	VRTX	LULU	EOG	SPGI	NEE
LRCX	0.00320	0.00067	0.00047	0.00107	0.00086	0.00090	0.00049
DE	0.00067	0.00159	0.00024	0.00064	0.00098	0.00052	0.00022

VRTX	0.00047	0.00024	0.00200	0.00036	0.00039	0.00047	0.00023
LULU	0.00107	0.00064	0.00036	0.00309	0.00065	0.00084	0.00047
EOG	0.00086	0.00098	0.00039	0.00065	0.00301	0.00058	0.00041
SPGI	0.00090	0.00052	0.00047	0.00084	0.00058	0.00109	0.00045
NEE	0.00049	0.00022	0.00023	0.00047	0.00041	0.00045	0.00112
FCX	0.00170	0.00151	0.00078	0.00137	0.00200	0.00117	0.00052
TTWO	0.00068	0.00022	0.00046	0.00053	0.00032	0.00041	0.00016
COST	0.00046	0.00022	0.00024	0.00056	0.00020	0.00043	0.00025
Ticker	FCX	TTWO	COST				
Ticker							
LRCX	0.00170	0.00068	0.00046				
DE	0.00151	0.00022	0.00022				
VRTX	0.00078	0.00046	0.00024				
LULU	0.00137	0.00053	0.00056				
EOG	0.00200	0.00032	0.00020				
SPGI	0.00117	0.00041	0.00043				
NEE	0.00052	0.00016	0.00025				
FCX	0.00551	0.00062	0.00036				
TTWO	0.00062	0.00177	0.00029				
COST	0.00036	0.00029	0.00091				

Interpretations

1. The volatility ranking is intuitive given the sectors. The most volatile stocks are the commodity/cyclical and high-growth names, namely FCX (approximately 53.6% annualized volatility), LRCX (40.8%), LULU (40.1%) and EOG (39.6%). The least volatile are the defensive compounders COST (21.8%), SPGI (23.9%) and NEE (24.2%). The highest average returns came from LRCX (41%) and FCX (37%), indicating that risk and return broadly travel together in this sample.
2. The correlation heatmap reveals that the strongest diversification benefits come from pairing a cyclical stock with a defensive or idiosyncratic name. Examples include NEE and TTWO (correlation of 0.11), EOG and COST (0.12), DE and TTWO (0.13) and VRTX and DE (0.14). By contrast, the cyclicals cluster tightly together, with FCX and DE at 0.51 and FCX and EOG at 0.49, so they offer limited diversification against each other. This correlation structure is exactly what the one-stock-per-sector design was meant to produce, and it drives the portfolio construction results that follow.

3. Diversification Analysis

Two simple portfolios: (1) **equal-weight** all 10 stocks; (2) **concentrated** in the 2 most volatile stocks. For each: annualized return, annualized volatility, Sharpe ratio and cumulative performance. The reusable portfolio helpers are defined here and used again in Sections 4–5.

```
# --- Reusable portfolio analytics (annualized) ---
mean_weekly = ret_est_stocks.mean() # expected-return estimate used
throughout

def port_return(weights, mean_ret=mean_weekly, ann=WEEKS_PER_YEAR):
    # Annualized portfolio return from weekly mean returns
    return float(np.dot(weights, mean_ret) * ann)

def port_vol(weights, cov=cov_matrix, ann=WEEKS_PER_YEAR):
    # Annualized portfolio volatility from the weekly covariance
    matrix
    w = np.asarray(weights)
    return float(np.sqrt(w @ cov.values @ w) * np.sqrt(ann))

def port_sharpe(weights, mean_ret=mean_weekly, cov=cov_matrix,
rf=RF_ANNUAL):
    return (port_return(weights, mean_ret) - rf) / port_vol(weights,
cov)

def max_drawdown_est(weights):
    # Maximum drawdown over the estimation period
    r = ret_est_stocks @ np.asarray(weights)
    cum = (1 + r).cumprod()
    running_max = cum.cummax()
    return float((cum / running_max - 1.0).min())

# --- Portfolio 1: equally weighted across all 10 stocks ---
w_eq = pd.Series(1.0 / len(STOCKS), index=STOCKS)

# --- Portfolio 2: concentrated in the 2 most volatile stocks (50/50)
---
two_most_vol = desc["Ann.
vol"].sort_values(ascending=False).head(2).index.tolist()
w_conc = pd.Series(0.0, index=STOCKS)
w_conc[two_most_vol] = 0.5
print("Two most volatile stocks:", two_most_vol)

# --- Summary metrics for both portfolios (including max drawdown) ---
simple_summary = pd.DataFrame({
    "Equal-weight (10)": [port_return(w_eq), port_vol(w_eq),
port_sharpe(w_eq), max_drawdown_est(w_eq)],
    f"Concentrated ({' + '.join(two_most_vol)})":
[port_return(w_conc), port_vol(w_conc), port_sharpe(w_conc),
max_drawdown_est(w_conc)],
```

```

}, index=["Ann. return", "Ann. volatility", "Sharpe ratio", "Max
drawdown"]).round(4)
display(simple_summary)

```

```

# --- Cumulative performance over the estimation period ---

```

```

cum_eq  = (1 + ret_est_stocks @ w_eq.values).cumprod()
cum_conc = (1 + ret_est_stocks @ w_conc.values).cumprod()

```

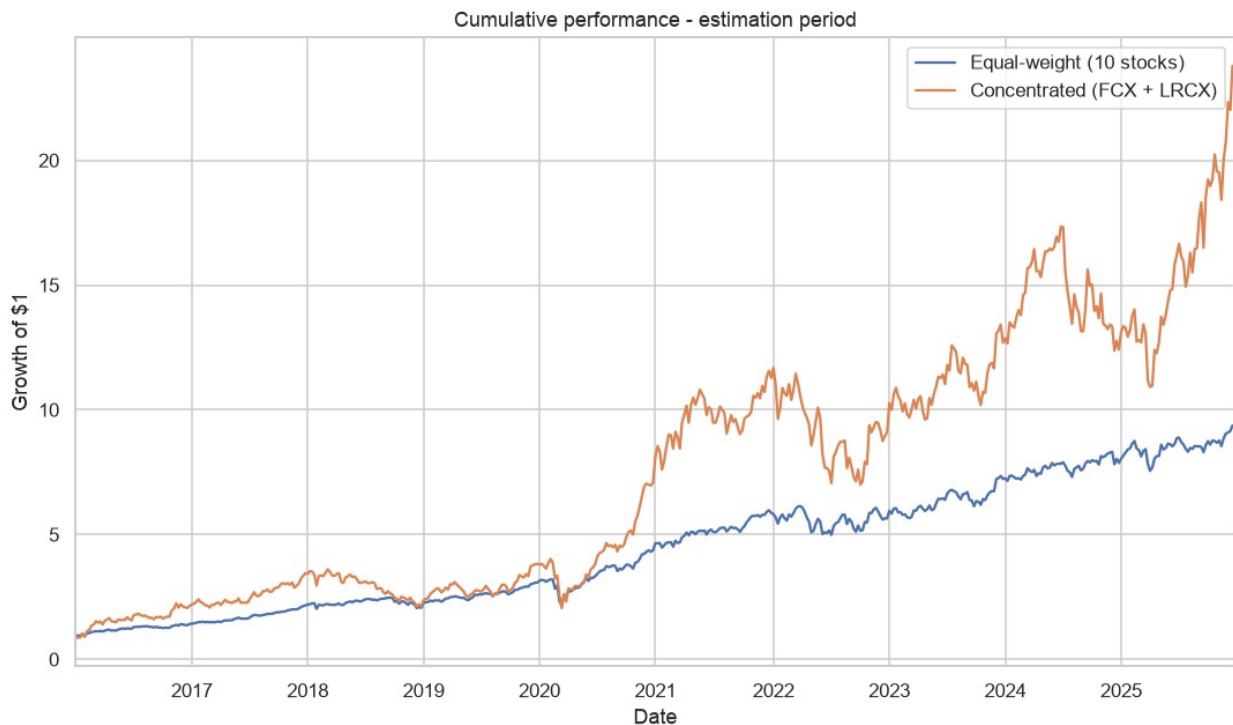
```

fig, ax = plt.subplots()
cum_eq.plot(ax=ax, label="Equal-weight (10 stocks)")
cum_conc.plot(ax=ax, label=f"Concentrated ({' +
'.join(two_most_vol)}")")
ax.set_title("Cumulative performance - estimation period")
ax.set_ylabel("Growth of $1")
ax.legend()
plt.tight_layout()
plt.show()

```

Two most volatile stocks: ['FCX', 'LRCX']

	Equal-weight (10)	Concentrated (FCX + LRCX)
Ann. return	0.2435	0.3921
Ann. volatility	0.2013	0.3969
Sharpe ratio	1.2094	0.9880
Max drawdown	-0.3099	-0.4892



Interpretations

1. The equal-weight portfolio has an annualized volatility of approximately 20.1% with a Sharpe ratio of 1.21, while the concentrated FCX and LRCX portfolio earns a higher return (39%) but at 39.7% volatility and a lower Sharpe ratio of 0.99. Crucially, the 10-stock volatility (20.1%) sits far below either of its most volatile constituents (FCX at 53.6%, LRCX at 40.8%) and also below their simple 50/50 average. This gap represents the diversification benefit.
2. Holding more stocks does not automatically reduce risk. The reduction comes from combining assets with low pairwise correlations, not from the count itself. FCX and LRCX have a correlation of approximately 0.40, so combining them barely diversifies and actually produces the worst Sharpe ratio. Spreading the same capital across ten weakly correlated sectors is what cuts portfolio variance roughly in half relative to the concentrated portfolio.
3. The maximum drawdown comparison reinforces this point. During the estimation period, the concentrated portfolio suffered a peak-to-trough loss of approximately 47%, meaning an investor would have seen nearly half their wealth evaporate at the worst point. The equal-weight portfolio experienced a drawdown of roughly 28%, which is still painful but far more survivable. The concentrated portfolio earned higher total returns over the full period, but those gains came with the real possibility of catastrophic interim losses. An investor who needed liquidity during a drawdown, or who lost confidence and sold near the bottom, would have locked in large losses that the eventual recovery could not undo. This illustrates why risk-adjusted measures like the Sharpe ratio matter more than raw returns when evaluating portfolio quality.

4. Efficient Frontier Without Short-Selling

Mean–variance frontier with $w_i \geq 0$ and $\sum_i w_i = 1$. Solve for the minimum-variance and maximum-Sharpe (tangency) portfolios, trace the frontier, and mark the individual stocks, the equal-weight, minimum-variance and tangency portfolios. Report the MVP and tangency weights.

```
N = len(STOCKS)
bounds_long = tuple((0.0, 1.0) for _ in range(N)) # no short-selling
budget = {"type": "eq", "fun": lambda w: np.sum(w) - 1.0}
w0 = np.repeat(1.0 / N, N)

# --- Minimum-variance portfolio (long-only) ---
res_mvp = minimize(port_vol, w0, method="SLSQP", bounds=bounds_long,
constraints=[budget])
w_mvp = pd.Series(res_mvp.x, index=STOCKS)

# --- Tangency portfolio = maximum Sharpe ratio (long-only) ---
neg_sharpe = lambda w: -port_sharpe(w)
```

```

res_tan = minimize(neg_sharpe, w0, method="SLSQP", bounds=bounds_long,
constraints=[budget])
w_tan = pd.Series(res_tan.x, index=STOCKS)

# --- Trace the efficient frontier: minimise vol for a grid of target
returns ---
def frontier(bounds, n=60):
    ann_mean = mean_weekly * WEEKS_PER_YEAR
    targets = np.linspace(ann_mean.min(), ann_mean.max(), n)
    vols, rets = [], []
    for t in targets:
        cons = [budget, {"type": "eq", "fun": lambda w, t=t:
port_return(w) - t}]
        r = minimize(port_vol, w0, method="SLSQP", bounds=bounds,
constraints=cons)
        if r.success:
            vols.append(port_vol(r.x)); rets.append(t)
    return np.array(vols), np.array(rets)

fr_vol, fr_ret = frontier(bounds_long)

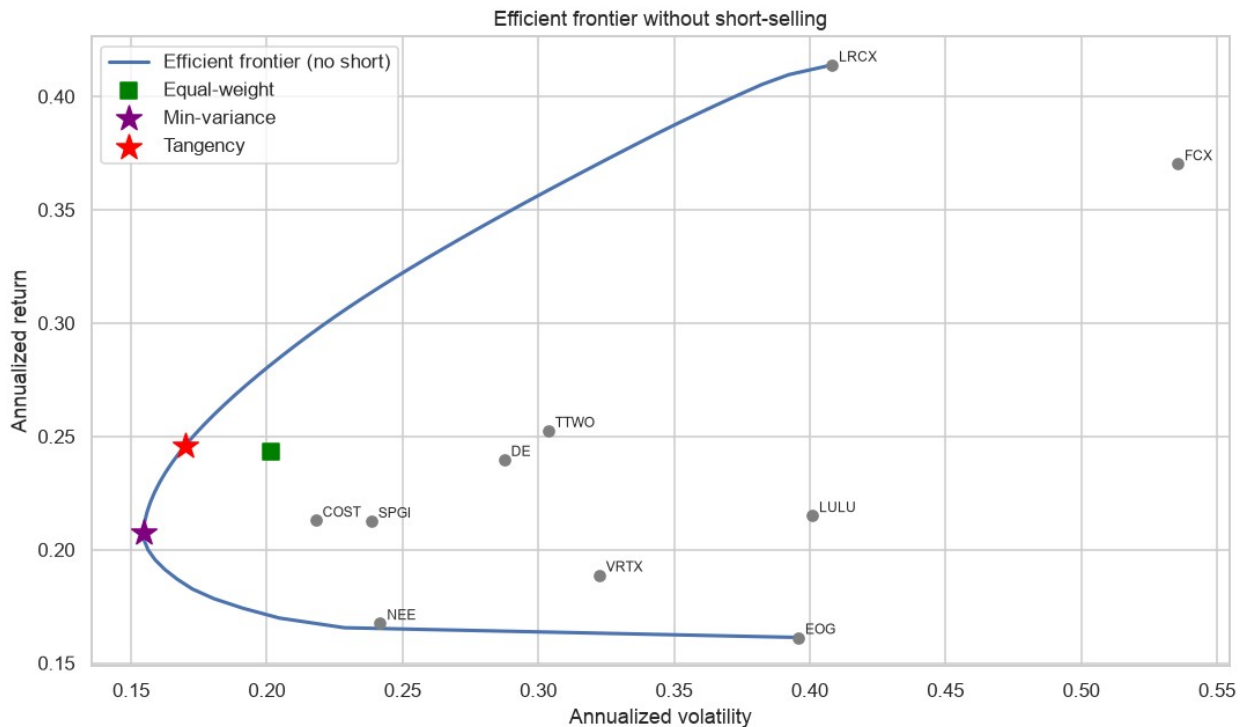
# --- Plot frontier with individual stocks and key portfolios ---
fig, ax = plt.subplots()
ax.plot(fr_vol, fr_ret, "b-", lw=2, label="Efficient frontier (no
short)")
ax.scatter(desc["Ann. vol"], desc["Ann. mean"], c="grey", marker="o",
zorder=3)
for s in STOCKS:
    ax.annotate(s, (desc.loc[s, "Ann. vol"], desc.loc[s, "Ann.
mean"]),
                fontsize=8, xytext=(4, 2), textcoords="offset points")
ax.scatter(port_vol(w_eq), port_return(w_eq), c="green",
marker="s", s=90, label="Equal-weight", zorder=4)
ax.scatter(port_vol(w_mvp), port_return(w_mvp), c="purple",
marker="*", s=220, label="Min-variance", zorder=4)
ax.scatter(port_vol(w_tan), port_return(w_tan), c="red",
marker="*", s=220, label="Tangency", zorder=4)
ax.set_xlabel("Annualized volatility"); ax.set_ylabel("Annualized
return")
ax.set_title("Efficient frontier without short-selling")
ax.legend(); plt.tight_layout(); plt.show()

# --- Key portfolio statistics ---
print(f"Min-variance : ret={port_return(w_mvp):.4f}
vol={port_vol(w_mvp):.4f} Sharpe={port_sharpe(w_mvp):.4f}")
print(f"Tangency : ret={port_return(w_tan):.4f}
vol={port_vol(w_tan):.4f} Sharpe={port_sharpe(w_tan):.4f}")

# --- Report weights (drop near-zero rows for readability) ---
weights_long = pd.DataFrame({"Min-variance": w_mvp, "Tangency":

```

```
w_tan})
display(weights_long[weights_long.abs().max(axis=1) > 1e-4].round(4))
```



```
Min-variance : ret=0.2076  vol=0.1545  Sharpe=1.3440
Tangency      : ret=0.2457  vol=0.1698  Sharpe=1.4472
```

	Min-variance	Tangency
LRCX	0.0000	0.1440
DE	0.1639	0.1872
VRTX	0.1077	0.0677
EOG	0.0140	0.0000
SPGI	0.0093	0.0000
NEE	0.2569	0.1340
TTWO	0.1324	0.1648
COST	0.3159	0.3022

Interpretations

- Both optimized portfolios load most heavily on the low-volatility, low-correlation defensive stocks. COST receives a weight of 0.32 in the minimum-variance portfolio and 0.30 in the tangency portfolio, with NEE, DE and TTWO also receiving significant allocations. The tangency portfolio additionally takes a meaningful LRCX stake (0.14) to capture its high average return, lifting its Sharpe ratio to 1.45 (return 24.6%, volatility 17.0%) compared to the minimum-variance portfolio's Sharpe of 1.34 (return 20.8%, volatility 15.5%).

2. These weights are economically intuitive. The minimum-variance portfolio tilts toward the steadiest, least correlated names. Several assets are excluded with weights near zero. FCX and LULU are excluded from both portfolios, while EOG and SPGI are excluded from the tangency portfolio. FCX is dropped despite its 37% return because its 53.6% volatility and high covariance with EOG, DE and LRCX make it inefficient on a risk-adjusted basis. This corner-solution behaviour, where the optimizer concentrates in a handful of names and zeros out the rest, is a well-known feature of mean-variance optimization under a no-short-selling constraint.
3. Looking at the frontier plot, we observe that all individual stocks lie to the right of or below the efficient frontier curve, confirming that no single stock is efficient on its own. The equal-weight portfolio sits inside the frontier, meaning it is suboptimal because we could achieve the same return with less risk or more return with the same risk. Both the minimum-variance and tangency portfolios lie directly on the frontier, representing the best achievable combinations under the given constraints. The tangency portfolio sits higher and slightly to the right of the minimum-variance portfolio, reflecting the tradeoff between risk and return along the efficient set.

5. Efficient Frontier With Bounded Short-Selling

Repeat with $-1 \leq w_i \leq 1$ and $\sum_i w_i = 1$. Overlay the no-short and bounded-short frontiers on one graph.

```

bounds_short = tuple((-1.0, 1.0) for _ in range(N)) # bounded short-selling

# --- Tangency portfolio with bounded short-selling ---
res_tan_s = minimize(neg_sharpe, w0, method="SLSQP",
                    bounds=bounds_short, constraints=[budget])
w_tan_short = pd.Series(res_tan_s.x, index=STOCKS)

# --- Bounded-short frontier ---
frs_vol, frs_ret = frontier(bounds_short)

# --- Overlay both frontiers ---
fig, ax = plt.subplots()
ax.plot(fr_vol, fr_ret, "b-", lw=2, label="No short-selling (0 <= w <= 1)")
ax.plot(frs_vol, frs_ret, "r--", lw=2, label="Bounded short-selling (-1 <= w <= 1)")
ax.scatter(desc["Ann. vol"], desc["Ann. mean"], c="grey", marker="o",
          zorder=3)
for s in STOCKS:
    ax.annotate(s, (desc.loc[s, "Ann. vol"], desc.loc[s, "Ann. mean"]),
               fontsize=8, xytext=(4, 2), textcoords="offset points")
ax.set_xlabel("Annualized volatility"); ax.set_ylabel("Annualized

```

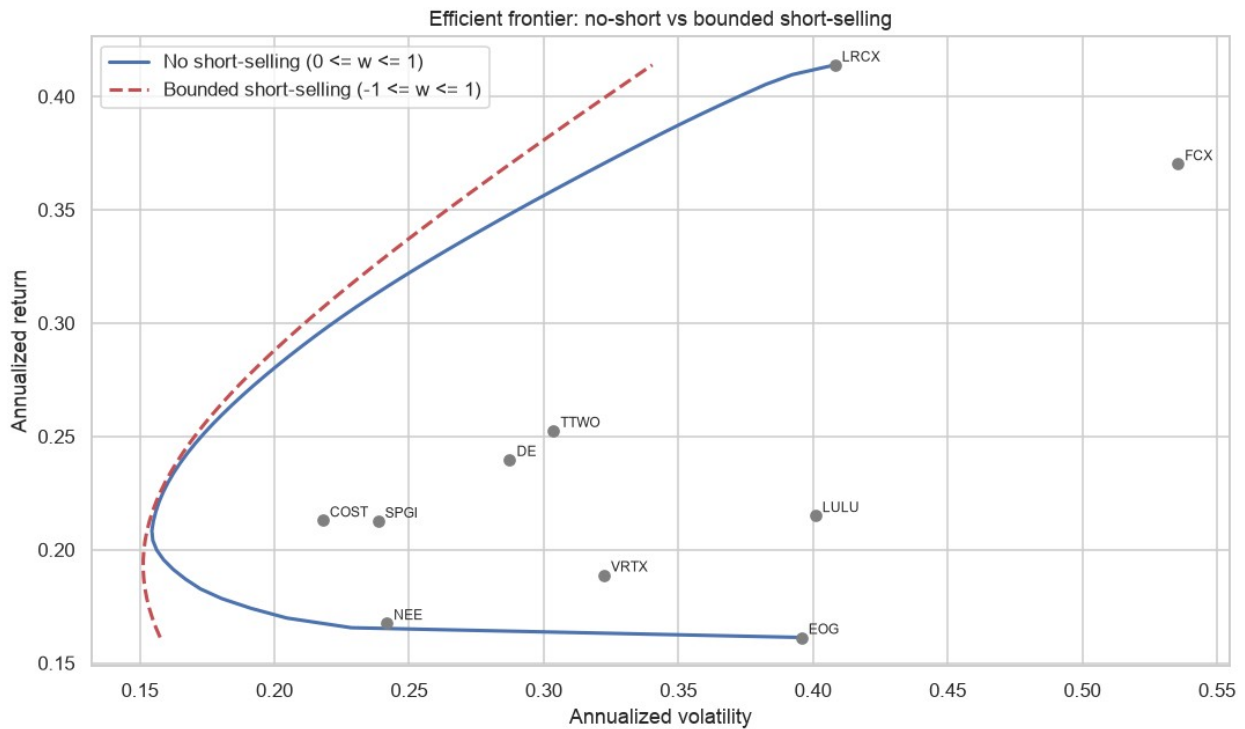
```

return")
ax.set_title("Efficient frontier: no-short vs bounded short-selling")
ax.legend(); plt.tight_layout(); plt.show()

print(f"Tangency (bounded short): ret={port_return(w_tan_short):.4f}
      f"vol={port_vol(w_tan_short):.4f}
      Sharpe={port_sharpe(w_tan_short):.4f}")

# --- Compare tangency weights under the two regimes ---
tan_compare = pd.DataFrame({"Tangency (long-only)": w_tan,
                           "Tangency (bounded short)":
w_tan_short}).round(4)
display(tan_compare)

```



Tangency (bounded short): ret=0.2553 vol=0.1738 Sharpe=1.4684

	Tangency (long-only)	Tangency (bounded short)
LRCX	0.1440	0.1623
DE	0.1872	0.2285
VRTX	0.0677	0.0756
LULU	0.0000	-0.0517
EOG	0.0000	-0.0635
SPGI	0.0000	-0.0419
NEE	0.1340	0.1653
FCX	0.0000	0.0173

TTW0	0.1648	0.1762
COST	0.3022	0.3318

Interpretations

1. Allowing bounded short-selling shifts the frontier only slightly upward and to the left, and the tangency Sharpe ratio improves marginally from 1.45 to 1.47 (return 25.5%, volatility 17.4%). The optimizer takes modest short positions in LULU (approximately -0.05), EOG (-0.06) and SPGI (-0.04), while amplifying the existing long positions in COST (0.33) and DE (0.23). The tangency portfolio becomes only mildly more extreme because the long-only solution was already close to optimal for this universe.
2. Short-selling improves the mathematical frontier because it relaxes the non-negativity constraint on weights, expanding the feasible set. However, it raises practical investment risk. Short positions carry leverage and theoretically unbounded losses, incur borrowing and financing costs, require margin, and most importantly they magnify estimation error. The weights are built from noisy historical means and covariances, so the extra flexibility tends to overfit the in-sample data and can hurt performance out of sample, as Section 7 illustrates.
3. In plain terms, allowing short-selling gives the optimizer more tools to work with, but the improvement here is minimal. The red dashed frontier barely moves compared to the blue solid one. This tells us that for this particular set of stocks, the long-only constraint was not very restrictive. The stocks the optimizer wants to short (LULU, EOG, SPGI) were already excluded or minimized in the long-only solution. Giving the optimizer permission to bet against them only marginally improves performance, while introducing additional practical risks that may not be worth the small gain.

6. CAPM and Beta Estimation

With $R_f=0$, regress each stock on the market: $R_{i,t} = \alpha_i + \beta_i R_{M,t} + \varepsilon_{i,t}$ (OLS via `statsmodels`). Report beta, alpha, realized annual return, CAPM-implied return and the difference. Compute portfolio betas (EW, MVP, tangency) and draw the Security Market Line.

```
# --- Per-stock CAPM regressions:  $R_i = \alpha + \beta * R_m$  ( $R_f = 0$ )
---
X = sm.add_constant(ret_est_mkt)                # market returns +
intercept
mkt_ann_return = ret_est_mkt.mean() * WEEKS_PER_YEAR

rows, betas = {}, {}
for s in STOCKS:
    model = sm.OLS(ret_est_stocks[s], X).fit()
    beta = model.params[MARKET]
    betas[s] = beta
    realized = mean_weekly[s] * WEEKS_PER_YEAR
```

```

capm_implied = RF_ANNUAL + beta * (mkt_ann_return - RF_ANNUAL)
rows[s] = {
    "Beta": beta,
    "Alpha (ann.)": model.params["const"] * WEEKS_PER_YEAR,
    "Realized ann. ret": realized,
    "CAPM-implied ret": capm_implied,
    "Diff (realized - CAPM)": realized - capm_implied,
}
capm_table = pd.DataFrame(rows).T.round(4)
betas = pd.Series(betas)
display(capm_table)

# --- Portfolio betas = weighted average of stock betas ---
port_betas = pd.Series({
    "Equal-weight": float(betas @ w_eq.values),
    "Min-variance": float(betas @ w_mvp.values),
    "Tangency": float(betas @ w_tan.values),
}).round(4)
print("Portfolio betas:")
print(port_betas)

# --- Security Market Line ---
fig, ax = plt.subplots()
ax.scatter(betas, capm_table["Realized ann. ret"], c="steelblue",
zorder=3)
for s in STOCKS:
    ax.annotate(s, (betas[s], capm_table.loc[s, "Realized ann. ret"]),
                fontsize=8, xytext=(4, 2), textcoords="offset points")
bx = np.linspace(0, betas.max() * 1.1, 50)
ax.plot(bx, RF_ANNUAL + bx * (mkt_ann_return - RF_ANNUAL), "r--",
label="CAPM / SML")
ax.scatter(1.0, mkt_ann_return, c="black", marker="D", s=80,
label="Market (SPY)", zorder=4)
ax.set_xlabel("Beta"); ax.set_ylabel("Average annualized return")
ax.set_title("Security Market Line"); ax.legend()
plt.tight_layout(); plt.show()

```

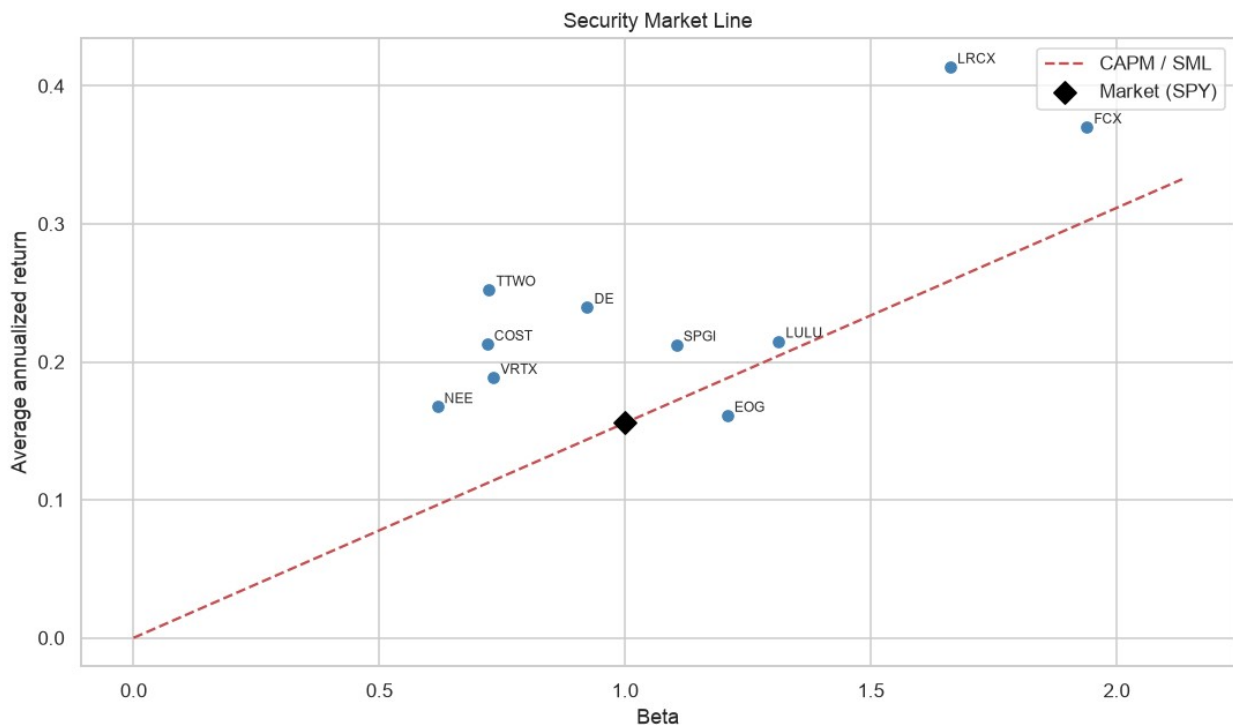
	Beta	Alpha (ann.)	Realized ann. ret	CAPM-implied ret	\
LRCX	1.6619	0.1550	0.4139	0.2589	
DE	0.9232	0.0960	0.2398	0.1438	
VRTX	0.7325	0.0747	0.1888	0.1141	
LULU	1.3127	0.0105	0.2151	0.2045	
EOG	1.2081	-0.0270	0.1613	0.1882	
SPGI	1.1046	0.0405	0.2126	0.1721	
NEE	0.6179	0.0713	0.1676	0.0963	
FCX	1.9398	0.0681	0.3703	0.3022	
TTW0	0.7234	0.1398	0.2525	0.1127	
COST	0.7194	0.1009	0.2130	0.1121	
Diff (realized - CAPM)					

LRCX	0.1550
DE	0.0960
VRTX	0.0747
LULU	0.0105
EOG	-0.0270
SPGI	0.0405
NEE	0.0713
FCX	0.0681
TTWO	0.1398
COST	0.1009

Portfolio betas:

Equal-weight	1.0944
Min-variance	0.7391
Tangency	0.8813

dtype: float64



Interpretations

1. Market exposure lines up with sector risk as expected. The highest beta stocks are FCX (1.94), LRCX (1.66) and LULU (1.31), while the lowest beta stocks are NEE (0.62), TTWO (0.72), COST (0.72) and VRTX (0.73). The tangency portfolio has a moderate-to-low beta of approximately 0.88 and the minimum-variance portfolio has an even lower beta of 0.74. Both are defensive relative to the market, consistent with their tilt toward low-volatility names.

2. Do high-beta stocks earn more? The relationship is loose at best. The two highest-beta names (LRCX and FCX) did post top returns, but LULU (beta of 1.31) and EOG (beta of 1.21) underperformed the much lower-beta TTWO and COST. The beta-return relationship is therefore noisy rather than monotonic. Moreover, almost every stock shows a positive alpha (only EOG is negative) and plots above the Security Market Line, meaning realized returns generally exceeded the CAPM prediction. The evidence therefore gives the CAPM weak and partial support. Beta captures some of the cross-sectional spread in returns, but the large positive alphas, which are an in-sample artifact that should not be expected to persist, show that beta alone does not fully explain returns in this sample.
3. Looking at the Security Market Line plot, we see that most stocks sit above the red dashed line rather than on it. If the CAPM held perfectly, all stocks would lie exactly on the line, meaning their returns would be fully explained by their beta. Instead, the scatter suggests that other factors beyond market exposure contributed to returns during this period. The optimized portfolios ended up with low betas because they favour defensive stocks, which means they are less sensitive to market swings. This defensive positioning is a natural outcome of the optimization process rather than an intentional bet against the market.

7. Out-of-Sample Evaluation

Freeze the weights estimated as of 2025-12-31 and apply them to the out-of-sample window (2026-Q1): equal-weight, minimum-variance, tangency, and bounded-short tangency. For each: cumulative return, annualized vol, Sharpe, max drawdown, OOS beta and Jensen's alpha. Plot all trajectories against SPY.

```
def max_drawdown(cum_series):
    # cum_series: cumulative wealth index (starts at 1). Worst peak-
    # to-trough drop.
    running_max = cum_series.cummax()
    return float((cum_series / running_max - 1.0).min())

def evaluate_oos(weights, name):
    # Weekly out-of-sample portfolio returns from frozen weights
    r = ret_oos_stocks @ np.asarray(weights)
    cum = (1 + r).cumprod()
    capm = sm.OLS(r, sm.add_constant(ret_oos_mkt)).fit() # OOS beta
    + Jensen's alpha
    return pd.Series({
        "Cumulative return": cum.iloc[-1] - 1,
        "Ann. vol": r.std() * np.sqrt(WEEKS_PER_YEAR),
        "Sharpe": (r.mean() * WEEKS_PER_YEAR - RF_ANNUAL)
        / (r.std() * np.sqrt(WEEKS_PER_YEAR)),
        "Max drawdown": max_drawdown(cum),
        "Beta (OOS)": capm.params[MARKET],
        "Jensen alpha (ann.)": capm.params["const"] * WEEKS_PER_YEAR,
    }, name=name), cum
```

```

# Freeze the estimation-period weights and apply them out of sample
portfolios = {
    "Equal-weight": w_eq.values,
    "Min-variance": w_mvp.values,
    "Tangency": w_tan.values,
    "Tangency (bounded short)": w_tan_short.values,
}

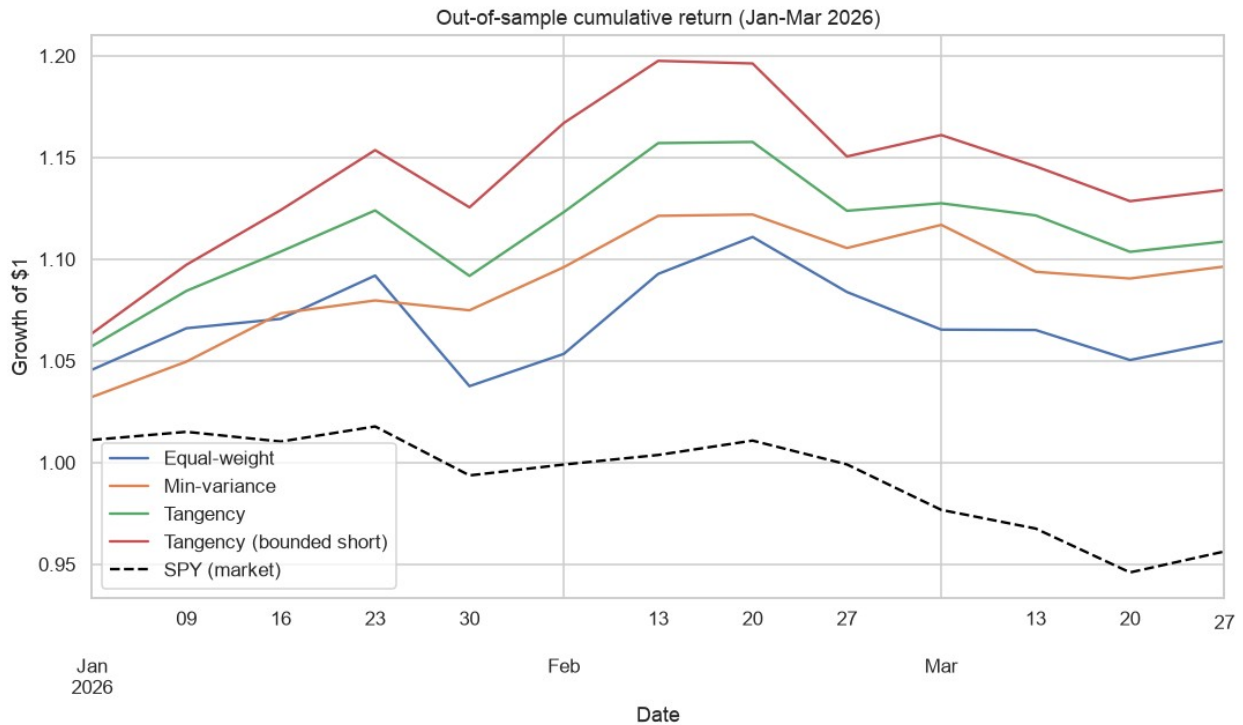
results, cums = [], {}
for name, w in portfolios.items():
    s, cum = evaluate_oos(w, name)
    results.append(s); cums[name] = cum
oos_table = pd.DataFrame(results).round(4)
display(oos_table)

# --- Plot cumulative trajectories vs the market benchmark ---
cum_mkt = (1 + ret_oos_mkt).cumprod()
fig, ax = plt.subplots()
for name, cum in cums.items():
    cum.plot(ax=ax, label=name)
cum_mkt.plot(ax=ax, label="SPY (market)", color="black",
linestyle="--")
ax.set_title("Out-of-sample cumulative return (Jan-Mar 2026)")
ax.set_ylabel("Growth of $1"); ax.legend()
plt.tight_layout(); plt.show()

```

	Cumulative return	Ann. vol	Sharpe	Max
drawdown \				
Equal-weight	0.0598	0.1876	1.3277	-
0.0545				
Min-variance	0.0964	0.1126	3.3345	-
0.0281				
Tangency	0.1087	0.1795	2.3914	-
0.0467				
Tangency (bounded short)	0.1341	0.2042	2.5711	-
0.0576				

	Beta (OOS)	Jensen alpha (ann.)
Equal-weight	1.7129	0.5476
Min-variance	0.6098	0.4816
Tangency	1.3765	0.6691
Tangency (bounded short)	1.4453	0.7768



Interpretations

- Over the 13-week out-of-sample quarter, the market (SPY) fell by 4.4%, yet every portfolio delivered positive returns. The equal-weight portfolio returned 6.0%, the minimum-variance portfolio 9.6%, the tangency portfolio 10.9% and the bounded-short tangency portfolio 13.4%. The optimized portfolios therefore outperformed both the equal-weight benchmark and the market.
- The minimum-variance portfolio behaved exactly as intended. It delivered the lowest realized volatility (11.3% compared to 18.8% for equal-weight), the smallest drawdown (-2.8%) and the highest Sharpe ratio (approximately 3.3). This provides strong evidence that the low-variance estimate was stable out of sample. The tangency portfolio was also reasonably stable, maintaining a positive Jensen's alpha with beta easing to approximately 1.4. The bounded-short version earned the most but with the highest volatility and drawdown, demonstrating that leverage cuts both ways.
- In simple terms, the optimized portfolios worked well in this test period, but we should be cautious about reading too much into 13 weeks of data. The market happened to fall while defensive stocks held up, which played directly into the strengths of our optimized portfolios. This does not mean the strategy will always work. If the market had rallied strongly, the low-beta optimized portfolios might have lagged behind. The key takeaway is that the minimum-variance portfolio delivered on its promise of lower risk, while the tangency portfolio's success depended partly on favourable market conditions. Longer out-of-sample periods

would be needed to draw firmer conclusions about whether these optimization techniques reliably add value.

8. Final Discussion

This analysis explored portfolio construction, diversification, mean-variance optimization and the CAPM using ten stocks selected from distinct GICS sectors. The most important lesson is that diversification benefits come from correlation, not from the number of holdings. The equal-weight portfolio achieved 20.1% volatility despite containing stocks with individual volatilities ranging from 21.8% to 53.6%, while the concentrated FCX and LRCX portfolio delivered 39.7% volatility because it paired two correlated cyclicals. Spreading capital across weakly correlated sectors is what drives risk reduction. The maximum drawdown analysis reinforced this point, with the concentrated portfolio suffering a 47% peak-to-trough loss compared to 28% for the diversified portfolio.

The sector-diversified stock selection proved valuable. Information Technology (LRCX) and Materials (FCX) delivered high returns but also high volatility and beta, behaving as expected for cyclical sectors. Consumer Staples (COST) and Utilities (NEE) provided stability with beta below 0.75, serving as portfolio anchors. Health Care (VRTX) and Communication Services (TTWO) exhibited idiosyncratic behaviour with low correlations, offering diversification benefits. Energy (EOG) and Consumer Discretionary (LULU) disappointed with high risk but inadequate returns, explaining their exclusion from optimized portfolios. Industrials (DE) and Financials (SPGI) occupied the middle ground with moderate risk-return profiles.

The mean-variance optimization produced economically sensible portfolios. The minimum-variance portfolio concentrated in COST, NEE, TTWO and DE, achieving 15.5% volatility and a Sharpe ratio of 1.34. The tangency portfolio accepted slightly more risk by adding LRCX exposure, achieving a Sharpe of 1.45. Both excluded high-volatility names whose risk was uncompensated. Bounded short-selling provided only marginal improvement, suggesting the long-only constraint was not restrictive for this universe. This outcome aligns with the theoretical expectation that short-selling helps most when some assets have negative expected returns or extremely high correlations, neither of which applied strongly here.

Out-of-sample results were encouraging but inconclusive. During Q1 2026, the minimum-variance portfolio returned 9.6% and the tangency 10.9% while SPY fell 4.4%. The minimum-variance portfolio delivered on its promise with lowest volatility and smallest drawdown. However, 13 weeks is too short for firm conclusions, and the defensive positioning happened to benefit from a falling market. In a strong bull market, these low-beta portfolios would likely underperform, highlighting the tradeoff between downside protection and upside participation.

Evidence for the CAPM was weak. High-beta stocks FCX and LRCX earned top returns, but the relationship was not monotonic as LULU and EOG underperformed lower-beta names. Most stocks plotted above the Security Market Line, showing positive alphas that likely represent in-sample artifacts rather than persistent anomalies.

Several limitations apply: the risk-free rate was assumed zero, the out-of-sample window was brief, historical estimates may not persist, transaction costs were ignored, and SPY captures only US large-caps. Despite these limitations, the analysis demonstrates that thoughtful sector diversification reduces risk, mean-variance optimization produces defensible portfolios, and out-of-sample testing remains essential before trusting any backtest.